

Name: Youssef Tarek Ismail Hefny

ID: 231027091

Assignment: Frequent Itemsets & Association Rules

Part 1: Apriori Algorithm

Minimum Support Count = 3

1. Generate C1 and Compute Support

We count the occurrences of each individual item in the 12 transactions:

- Bread: 10
- Milk: 8
- Diapers: 7
- Beer: 6
- Butter: 5
- Eggs: 2

2. Generate L1

We prune items where support < 3:

- **L1:** Bread: 10, Milk: 8, Diapers: 7, Beer: 6, Butter: 5}

3. Generate C2 and L2 (Sample of Frequent Pairs)

We combine L1 items into pairs and check support:

- {Milk, Bread}: 7
- {Bread, Diapers}: 6
- {Beer, Diapers}: 5
- {Milk, Butter}: 3

4. Generate C3 and L3 (Frequent Triplets)

- {Milk, Bread, Diapers}: 4
- {Bread, Diapers, Beer}: 4
- {Milk, Bread, Butter}: 3

Part 2: Association Rules

Minimum Confidence = 60%

1. Strong Rules Generation

Based on the frequent itemsets, we calculate Confidence and Lift:

Rule	Support	Confidence	Lift	Interpretation
{Beer} → {Diapers}	5/12	83.3%	1.43	Positive Correlation
{Butter} → {Bread}	4/12	80%	0.96	Nearly Independent
{Milk, Diapers} → {Bread}	4/12	80%	0.96	Strong Confidence

Part 3: FP-Growth Algorithm

Minimum Support = 3

1. **Frequency Table:** Items are ordered by frequency: Bread (10), Milk (8), Diapers (7), Beer (6), Butter (5).
2. **FP-Tree Construction:** A prefix tree is built by mapping transactions in the order of the frequency table. This avoids the need for candidate generation.
3. **Frequent Itemsets:** The results match the Apriori output, showing {Bread, Milk}, {Bread, Diapers}, and {Beer, Diapers} as core patterns.

Part 4: Comparison & Reflection

1. **Comparison:** Apriori uses a bottom-up approach and generates candidate sets, whereas FP-Growth uses a tree structure to mine patterns without candidates.
2. **Scans:** Apriori performs multiple scans (one for each itemset size), while FP-Growth only scans the database **twice**.
3. **Findings:**
 - **Most Frequent Item:** Bread (Support = 10).
 - **Strongest Pair:** Beer, Diapers} due to high Lift and Confidence.
4. **Candidate Explosion:** FP-Growth avoids explosion because it compresses the database into an FP-Tree, allowing it to find frequent patterns by partitioning the tree instead of testing every combination

Python Code (Solution Script)

```
import pandas as pd
from mlxtend.preprocessing import TransactionEncoder
from mlxtend.frequent_patterns import apriori, association_rules

# Data loading
transactions = [
    ['Milk', 'Bread', 'Butter'], ['Bread', 'Diapers', 'Beer', 'Eggs'],
    ['Milk', 'Bread', 'Diapers', 'Beer'], ['Bread', 'Milk', 'Diapers', 'Beer'],
    ['Milk', 'Bread', 'Butter'], ['Bread', 'Butter'],
    ['Milk', 'Diapers', 'Beer'], ['Milk', 'Bread', 'Diapers', 'Butter'],
    ['Bread', 'Diapers', 'Beer'], ['Milk', 'Bread', 'Beer'],
    ['Milk', 'Bread', 'Diapers'], ['Bread', 'Butter', 'Eggs']
]

# Encoding
te = TransactionEncoder()
te_ary = te.fit(transactions).transform(transactions)
df = pd.DataFrame(te_ary, columns=te.columns_)

# Apriori
freq_itemsets = apriori(df, min_support=3/12, use_colnames=True)

# Rules
rules = association_rules(freq_itemsets, metric="confidence", min_threshold=0.6)
print(rules[['antecedents', 'consequents', 'support', 'confidence', 'lift']])
```

✓ 0.0s

Output :

...	antecedents	consequents	support	confidence	lift
0	(Beer)	(Bread)	0.416667	0.833333	0.909091
1	(Diapers)	(Beer)	0.416667	0.714286	1.428571
2	(Beer)	(Diapers)	0.416667	0.833333	1.428571
3	(Beer)	(Milk)	0.333333	0.666667	1.000000
4	(Butter)	(Bread)	0.416667	1.000000	1.090909
5	(Diapers)	(Bread)	0.500000	0.857143	0.935065
6	(Bread)	(Milk)	0.583333	0.636364	0.954545
7	(Milk)	(Bread)	0.583333	0.875000	0.954545
8	(Butter)	(Milk)	0.250000	0.600000	0.900000
9	(Diapers)	(Milk)	0.416667	0.714286	1.071429
10	(Milk)	(Diapers)	0.416667	0.625000	1.071429
11	(Diapers, Bread)	(Beer)	0.333333	0.666667	1.333333
12	(Diapers, Beer)	(Bread)	0.333333	0.800000	0.872727
13	(Beer, Bread)	(Diapers)	0.333333	0.800000	1.371429
14	(Beer)	(Diapers, Bread)	0.333333	0.666667	1.333333
15	(Beer, Bread)	(Milk)	0.250000	0.600000	0.900000
16	(Beer, Milk)	(Bread)	0.250000	0.750000	0.818182
17	(Diapers, Beer)	(Milk)	0.250000	0.600000	0.900000
18	(Diapers, Milk)	(Beer)	0.250000	0.600000	1.200000
19	(Beer, Milk)	(Diapers)	0.250000	0.750000	1.285714
20	(Butter, Bread)	(Milk)	0.250000	0.600000	0.900000
21	(Butter, Milk)	(Bread)	0.250000	1.000000	1.090909
22	(Butter)	(Bread, Milk)	0.250000	0.600000	1.028571
23	(Diapers, Bread)	(Milk)	0.333333	0.666667	1.000000
24	(Diapers, Milk)	(Bread)	0.333333	0.800000	0.872727